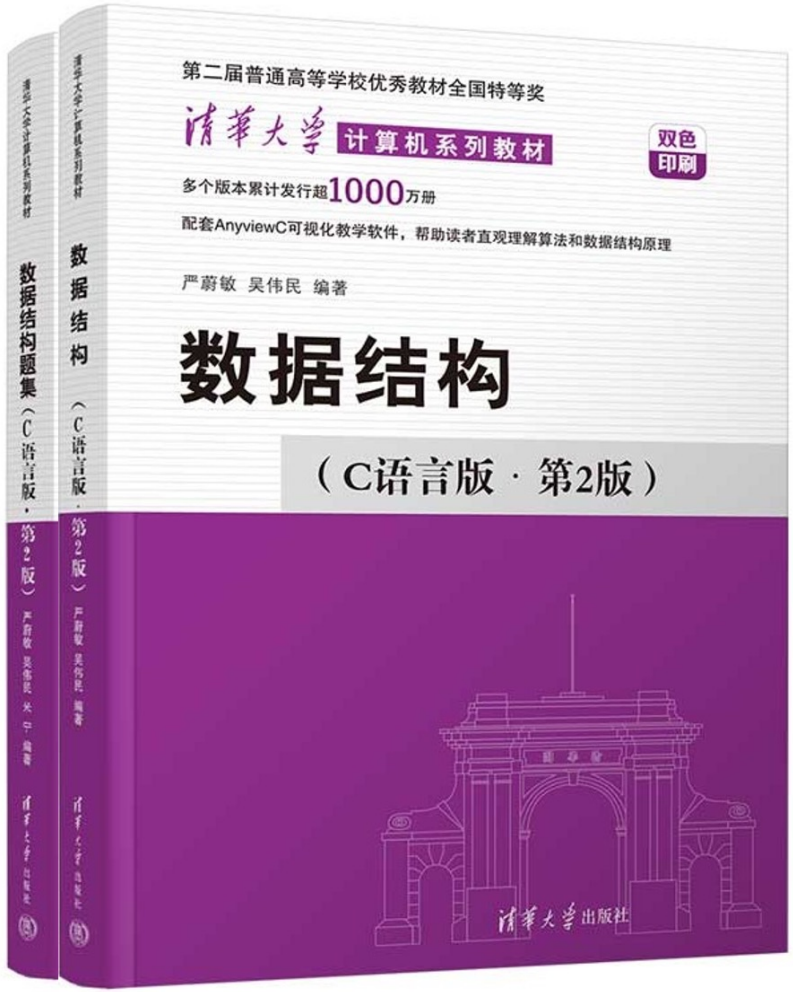


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第5章

数组和广义表

5.1 数组的定义

类似于线性表，抽象数据类型数组可形式地定义为：

ADT Array {

数据对象： D = { $a_{j1j2\cdots jn}$ | n（>0）称为数组的维数，b_i是数组第i维的长度，
$j_i = 0,\cdots,b_i-1$，i=1,2,…,n，
j_i是数组元素的第i维下标，$a_{j1j2\cdots jn} \in$ElemSet }

数据关系： R = { R_1, R_2,…, R_n }

R_i = { < $a_{j1\cdots ji\cdots jn}$, $a_{j1\cdots ji+1\cdots jn}$ > |
$0 \leqslant j_k \leqslant b_k-1$，$1 \leqslant k \leqslant n$且$k \neq i$，
$0 \leqslant j_i \leqslant b_i-2$，
$a_{j1\cdots ji\cdots jn}$, $a_{j1\cdots ji+1\cdots jn} \in D$，i=2,…,n }

基本操作：

NewArray(n, bound1,…, boundn);

操作结果：若维数n和各维长度合法，则构建并返回相应的数组。

FreeArray(A);

操作结果：回收数组A，须重新构建才能作为数组使用，返回NULL。

PutElem(A, e, index1,…, indexn);

初始条件：A是n维数组，e为元素，随后是n个下标值。

操作结果：若下标不超界，则将e的值赋给所指定的A的元素，并返回OK。

GetElem(A, indexl,…, indexn);

初始条件：A是n维数组，随后是n个下标值。

操作结果：若各下标不超界，则返回所指定的A的元素值。

} ADT Array

5.2 数组的顺序表示和实现

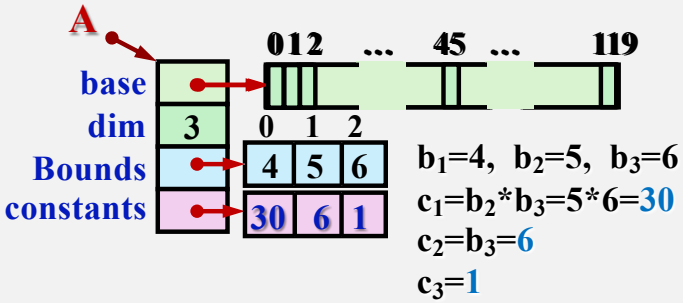
清華大學出版社 TSINGHUA UNIVERSITY PRESS

多维数组的访问特性决定了应该采用顺序存储结构，但可以有不同的具体方案。

这里讨论一种为数组元素分配连续存储空间的方案。

```
//----- 数组的顺序存储表示 -----
#include <stdarg.h>   // 标准头文件，
      // 提供宏va_start、va_arg和va_end，
      // 用于获取变长参数表的实际参数
#define  MAX_ARRAY_DIM  8   // 设定维数最大值为 8
typedef struct {
  ElemType *base;    // 元素基址，由InitArray分配
  int    dim;        // 数组维数
  int   *bounds;     // 维界的基址，由InitArray分配
  int   *constants;  // 数组映象函数的常数表基址，
                     // 由InitArray分配
} *Array;    // 多维数组类型
```

例 `int a[4][5][6];` 的存储结构 Array A；



元素 a[1][2][3]的存储位置：

A->base + c_1 *1 + c_2*2 + c_3*3 =

A->base + 30*1 + 6*2 + 1*3 = A->base + 45

即： A->base[45]

多维数组的顺序存储表示和实现

C语言的变长参数表。定义时，用三个圆点“.”表示。
调用时，多个逗号分隔的实际参数。
与此配套的是可变参数表类型va_list及其三个宏。

ap
3 4 5 6

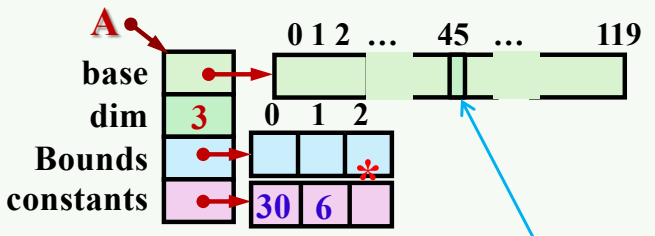
```
Array NewArray(int dim, ...) { // 构造多维数组A存储结构
  if(dim<1 || dim>MAX_ARRAY_DIM) return ERROR;
  Array A;   int i, elemtotal = 1;  // 数组元素个数的初值
  if(!(A = (Array)malloc(sizeof(*A))))exit(OVERFLOW);
  va_list ap;   A->dim = dim;     // 维数
  if(!(A->bounds = (int*)malloc(dim*sizeof(int))))exit(OVERFLOW);
     // 按维数分配各维长度的存储空间
     // 若各维长度合法，则存入A.bounds，并求出A的元素总数elemtotal
  va_start(ap, dim);     // ap为va_list类型，是存放变长参数表信息的数组
  for(i=0; i<dim; ++i) {  // 从变长参数表获取各维长度，并计算元素总数
    A->bounds[i]=va_arg(ap,int);  // 从变长参数表获取维的长度
    if(A->bounds[i]<0)return UNDERFLOW; // 维的长度不能小于0
    elemtotal *= A->bounds[i];    // 累乘求元素总数
  }
  va_end(ap);    // 变长参数获取结束，释放资源
  if(!(A->base=(ElemType*)calloc(elemtotal,sizeof(ElemType))))exit(OVER
        // 求映象函数的常数ci,1并存入A.constants[i-1], i=1,...,dim
  if(!(A->constants=(int*)malloc(dim*sizeof(int))))exit(OVERFLOW); // 按
  A->constants[dim-1] = 1;       // 常数c1
  for(i=dim-2; i>=0; --i) A->constants[i]=A->bounds[i+1]*A->constants[i+1];  // 元素地址计算常数
  return A;
}
```

120 456

例 int a[4][5][6];

调用：InitArray(3, 4, 5, 6);

构建了 Array a; 的存储结构：



元素 a[1][2][3]的存储位置：

$a.base + c_1*1 + c_2*2 + c_3*3 =$

$a.base + 30*1 + 6*2 + 1*3 = a.base + 45$

即： a.base[45]

多维数组元素的取值与赋值

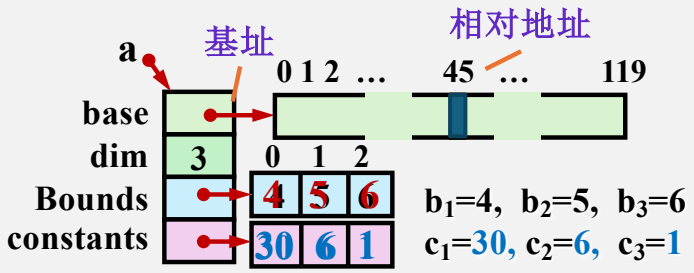
多维数组最常用的基本操作是对元素的取值和赋值：

```
Status PutElem(Array A,  ElemType e, ... );
ElemType GetElem(Array A, ... );
```

如构造算法，利用变长参数表，统一算法对不同维数的接口。

首先讨论两个操作都调用的计算元素相对地址的CalcOff操作。

例 int a[4][5][6]; 的存储结构 Array a;



返回相对地址　　已初始化的变长参数表

```
int CalcOff(Array A, va_list ap) {
   // 若ap指示的各下标值合法，
   // 则求出该元素在多维数组A中相对地址off
   int idx, off = 0;    // 相对地址初值
   for (int i=0; i<A->dim; i++) {
      idx = va_arg(ap, int);  // 获取下标值
      if (idx<0 || idx>=A->bounds[i]) return -1;
      off += A->constants[i] * idx; // 迭代计算相对地址
   }
   return Off;
}
```

元素 a[1][2][3]的存储位置：

A->base+ c_1 *1 + c_2*2 + c_3*3 =

A->base+ 30*1 + 6*2 + 1*3 = A->base+45

即： A->base[45]

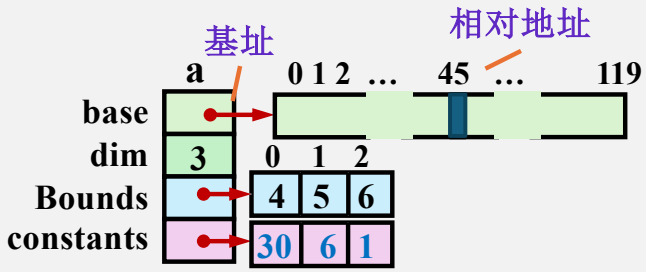
多维数组元素的取值与赋值

清華大學出版社 TSINGHUA UNIVERSITY PRESS

对三维数组元素 a[1][2][3]取值，调用基本操作：

GetElem(a, 1, 2, 3);

例 int a[4][5][6]; 的存储结构 Array a;




```
ElemType GetElem(Array A,...) {
    // A是n维数组，随后是n个下标值。
    // 若各下标不超界，则返回所指定的A的元素值
    if (!A) return errV;
    va_list ap;    va_start(ap, A);   // 变长参数表初始化
    int off;
    if ((off = CalcOff(A, ap)) < 0) return errV;
    return *(A->base + off);
}
```

45 返回的相对地址

元素 a[1][2][3]的存储位置：

A->base + 45

即： A->base[45]

多维数组元素的取值与赋值

对三维数组元素a[1][2][3]赋值，调用基本操作：

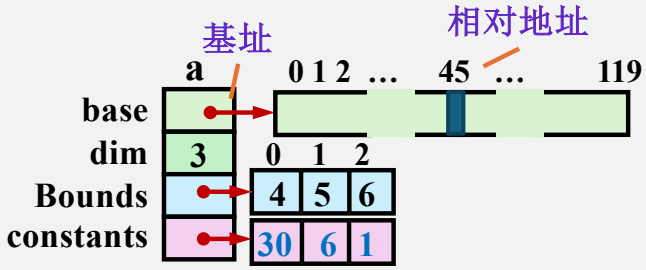
```
PutElem(a, ev, 1, 2, 3);
```

```
Status Assign(Array A,  ElemType e, ...) {
   // A是n维数组，e为元素变量，随后是n个下标值。
   // 若下标不超界，则将e的值赋给A的指定元素，并返回OK。
   if (!A) return ERROR;
   va+list ap;    va_start(ap, e);
   int off;
   if ((off = CalcOff(A, ap)) < 0) return ERROR;
   *(A->base + off) = e;
   return OK;
}
```

45 返回相对地址

赋值

例 int a[4][5][6]; 的存储结构 Array a;



元素 a[1][2][3]的存储位置：

A->base + 45

即： A->base[45]

5.3 矩阵的压缩存储

5.3.1 特殊矩阵

若n阶矩阵A中的元满足下述性质

$$a_{ij} = a_{ji} \quad 1 < i, j \leqslant n$$

则称为n阶**对称矩阵**。

对于对称矩阵，可以为每一对**对称元**分配一个存储空间，则可将n^2个元**压缩存储**到$n(n+1)/2$个元的空间中。不失一般性，以行序为主序存储其**下三角**（包括对角线）中的元。

假设以一维数组 $sa[n(n+1)/2]$ 作为n阶对称矩阵A的存储结构，则$sa[k]$和矩阵元a_{ij}之间存在着一一对应的关系：

$$k = \begin{cases} \dfrac{i(i-1)}{2} + j - 1 & \text{当} i \geq j \\ \dfrac{j(j-1)}{2} + i - 1 & \text{当} i \leq j \end{cases} \qquad (5-3)$$

对任意给定一组下标（i, j），均可在sa中找到矩阵元a_{ij}，反之，对所有的$k = 0, 1, 2, \ldots, n(n+1)/2-1$，都能确定$sa[k]$中的元在矩阵中的位置（$i, j$）。由此，称$sa[n(n+1)/2]$为$n$阶**对称矩阵**$A$的压缩存储（见图5.3）。

	a_{11}	a_{21}	a_{22}	a_{31}	…	a_{n1}	…	a_{nn}
$k=$	0	1	2	3		$\dfrac{n(n-1)}{2}$		$\dfrac{n(n+1)}{2}-1$

三角矩阵和对角矩阵

压缩存储的方法同样也适用于**三角矩阵**。所谓**下（上）三角矩阵**是指矩阵的上（下）三角（不包括对角线）中的元均为常数c或零的n阶矩阵。则除了和对称矩阵一样，只存储其下（上）三角中的元之外，再加一个存储常数c的存储空间即可。

在数值分析中经常出现的还有另一类特殊矩阵是**对角矩阵**。在这种矩阵中，所有的非零元都集中在以主对角线为中心的**带状区域**中。即除了主对角线上和直接在对角线上、下方若干条对角线上的元之外，所有其他的元皆为零。如图5.4所示。对这种矩阵，也可按某个原则（或以行为主，或以对角线的顺序）将其压缩存储到一维数组上。

在统称为特殊矩阵的这些矩阵中，非零元的分布都有明显规律，都可将其压缩存储到一维数组中，并找到每个非零元在一维数组中的对应关系。

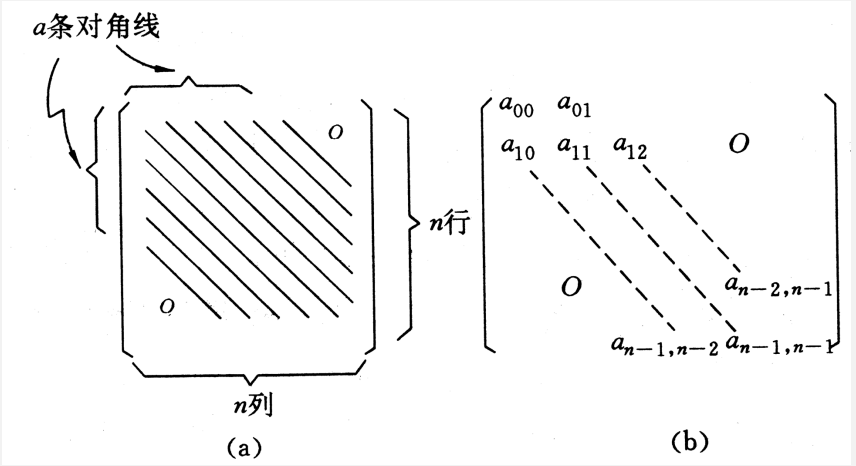


图5.4 对角矩阵
(a) 一般情形； (b)三对角矩阵

5.3.2 稀疏矩阵

什么是稀疏矩阵？没有严格的定义，只是一个大致的表述。假设在$m \times n$矩阵中，有t个元素不为零。

令 $\delta = \frac{t}{m \times n}$，称$\delta$为矩阵的**稀疏因子**。通常认为$\delta \leq 0.05$时称为稀疏矩阵。矩阵运算的种类很多，在稀疏矩阵的下列抽象数据类型定义中，只列举了几种常见的运算。

ADT SparseMat {

数据对象：**D = {** $a_{i,j}$ | i=1,2,…,m; j=1,2,…,n;
$a_{i,j}$ ∈ElemSet，m和n分别称为矩阵的行数和列数 **}**

数据关系：**R = {** Row, Col **}**

Row = { < $a_{i,j}$, $a_{i,j+1}$ > | 1≤i≤m, 1≤j≤n-1 **}**

Col = { < $a_{i,j}$, $a_{i+1,j}$ > | 1≤i≤m-1, 1≤j≤n **}**

基本操作：

NewSMat();
操作结果：创建并返回稀疏矩阵。

FreeSMat(M);
初始条件：稀疏矩阵M存在。
操作结果：回收稀疏矩阵M。

PrintSMat(M);
初始条件：稀疏矩阵M存在。
操作结果：输出稀疏矩阵M。

CopySMat(M);
初始条件：稀疏矩阵M存在。
操作结果：返回由M复制得到稀疏矩阵。

AddSMat(M, N);
初始条件：稀疏矩阵M与N的行数和列数对应相等。
操作结果：求并返回稀疏矩阵的和M+N。

SubSMat(M, N);
初始条件：稀疏矩阵M与N的行数和列数对应相等。
操作结果：求并返回稀疏矩阵的差M-N。

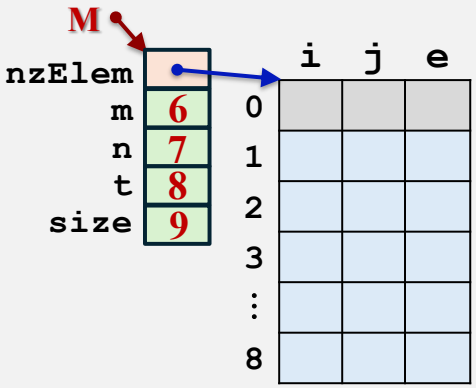
MultSMat(M, N);
初始条件：稀疏矩阵M的列数等于N的行数。
操作结果：求并返回稀疏矩阵乘积M×N。

TransposeSMat(M);
初始条件：稀疏矩阵M存在。
操作结果：求并返回稀疏矩阵M的转置矩阵。

} ADT SparseMat

1. 三元组顺序表

以三元组作为顺序表的元素类型，就得到稀疏矩阵的一种压缩存储方式——**三元组顺序表**，以下是其类型定义。存储分配辅助操作**allocTSMat**为一个含有**t**个非零元的**m×n**矩阵分配存储空间，但未存入非零元三元组。由一个二维数组存储的稀疏矩阵构造其三元组顺序表或者在矩阵操作中构造三元组顺序表，都可以调用**allocTSMat**分配存储空间。




```
typedef struct {
   int      i, j;    // 该非零元的行下标和列下标
   ElemType e;       // 非零元
} Triple;            // 三元组类型
typedef struct {
   Triple *nzElem;  // 动态分配的非零元三元组表. nzElem[0]未用
   int     m, n, t; // 矩阵的行数、列数和非零元个数
   int     size;    // 三元组表容量
} *TSMat;           // 三元组顺序表指针类型

TSMat allocTSMat(int m, int n, int t) {  // 存储分配辅助操作（未存储非零元）
    if (m<1 || n<1 || t<0) return NULL;   // 参数不合法则返回NULL
    TSMat M;                              // nzElem中的非零元三元组以行序为主序顺序排列
    if (!(M = (TSMat)malloc(sizeof(*M)))) exit(OVERFLOW);    // 分配结构记录和三元组表空间
    if (!(M->nzElem = (Triple*)malloc((size+1)*sizeof(Triple)))) exit(OVERFLOW);
    M->m = m;   M->n = n;   M->t = t;   M->size = t+1;      // 置行、列、三元组数和容量
    return M;
}
```

6 7 8

三元组表存储的稀疏矩阵的转置

对于$m \times n$矩阵**M**，它的**转置矩阵**T是n×m矩阵，且$T(i, j) = M(j, i)$，$1 \le i \le n$, $1 \le j \le m$。图5.5中的矩阵**M**和**T**互为转置矩阵。

一个**稀疏矩阵**的**转置矩阵**仍然是**稀疏矩阵**。假设**a**和**b**是**TSMat**型的变量，分别表示矩阵**M**和**T**。那么，如何由**a**得到**b**呢？

从分析**a**和**b**之间的差异可见，只要做到以下三点：

（1）将矩阵的行列值相互**交换**；

（2）将每个三元组中的i和j相互**调换**；

（3）**重排**三元组之间的次序便可实现矩阵的转置。

前二点是容易做到的，关键是如何实现第三点。即如何使**b->nzElem**中的三元组是以**T**的行（**M**的列）**按主序依次排列**的。

$$M=\begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix} \qquad T=\begin{bmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{bmatrix}$$

M **T**

图5.5 稀疏矩阵M和T——互为转置矩阵

a->nzElem

下标	i	j	e
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

b->nzElem

下标	i	j	e
1	1	3	-3
2	1	6	15
3	2	1	12
4	2	5	18
5	3	1	9
6	3	4	24
7	4	6	-7
8	6	3	14

稀疏矩阵M和T分别存储为三元组表a和b

（1）算法5.3 —— 常规转置方法

对b->nzElem中三元组依次在a->nzElem中找到相应的三元组进行转置。即，按照矩阵M的列序进行转置。

为了找到M的每一列中所有的非零元素，需要对其三元组表a->nzElem从第一行起整个扫描一遍，

由于a->nzElem是以M的行序为主序来存放每个非零元的，由此得到的恰是b->nzElem应有的顺序。

```
TSMat TransposeTSMat(TSMat M) { // 求并返回稀疏矩阵M的转置矩阵
  if (!M) return NULL;          // 若M不存在，则返回NULL报错
  TSMat T = allocTSMat(M->n, M->m, M->t);
  if (T->t) {
    int p, q=1, col;
    for (col=1; col<=M->n; ++col) // 按列号从小到复制非零元
      for (p=1; p<=M->t; ++p)     // 依次扫描三元组
        if (M->nzElem[p].j == col) { // 若是当前列号的非零元
          T->nzElem[q].i = M->nzElem[p].j;  // 复制三元组
          T->nzElem[q].j = M->nzElem[p].i;
          T->nzElem[q].e = M->nzElem[p].e;
          q++; // q指示转置矩阵的三元组表末尾加入位置
        }
  }
  return T;    // 返回转置矩阵
}
```

下标	i	j	e
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

a->nzElem

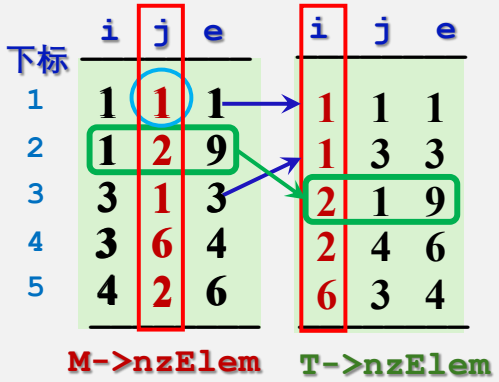
下标	i	j	e
1			
2			
3			
4			
5			
6			
7			
8			

b->nzElem

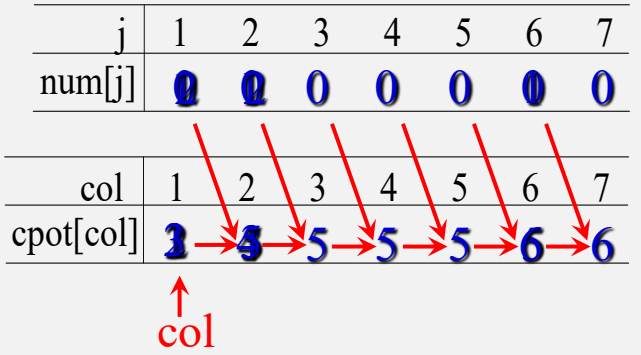
（2）算法5.4 —— 快速转置方法

1. 矩阵T的初始化
2. 统计num数组
3. 计算cpot数组
4. 元素转置

分析每列非零元转置位置




```
TSMat FastTransposeSMat(TSMat M) {
  if (!M) return NULL;
  int col, t, p, q, num[20], cpos[20];
  TSMat T = allocTSMat(M->n, M->m, M->t);
  if (T->t!=0) {
    for (col=1; col<=M->n; ++col) num[col] = 0;
    for (t=1; t<=M->t; ++t) ++num[M->nzElem[t].j];
    cpot[1] = 1;
    for (col=2; col<=M->n; ++col)
      cpot[col] = cpot[col-1] + num[col-1];
    for (p=1; p<=M->t; ++p) {
      col = M->nzElem[p].j;   q = cpot[col];
      T->nzElem[q].i = M->nzElem[p].j;
      T->nzElem[q].j = M->nzElem[p].i;
      T->nzElem[q].e = M.data[p].e;
      ++cpot[col] ;
    }
  }
  return T;
}
```



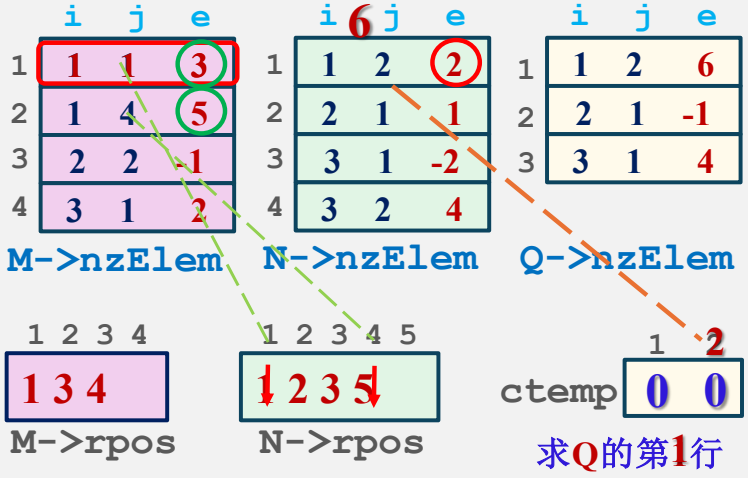
算法5.5 稀疏矩阵相乘

$$M=\begin{bmatrix}3 & 0 & 0 & 5\\ 0 & -1 & 0 & 0\\ 2 & 0 & 0 & 0\end{bmatrix}\quad N=\begin{bmatrix}0 & 2\\ 1 & 0\\ -2 & 4\\ 0 & 0\end{bmatrix}\quad Q=\begin{bmatrix}0 & 6\\ -1 & 0\\ 0 & 4\end{bmatrix}$$

$$M \times N = Q$$

矩阵乘法规则：$q_{ij} = \sum_{k=1}^{n_1} m_{ik} \times nkj$

稀疏矩阵采用行逻辑链接顺序表存储结构，
需要相应的求q_{ij}的方法。



$3 \times 0 = 0$
$0 \times 1 = 0$
$0 \times -2 = 0$
$5 \times 0 = 0$ } 0

$3 \times 2 = 6$
$0 \times 0 = 0$
$0 \times 4 = 0$
$5 \times 0 = 0$ } 6

分析

（1）稀疏矩阵乘法用常规二维数组求行和列点积方法，会做大量积为0的无效乘法

（2）存储结构决定了不能用二维数组求行和列点积方法。

基于行逻辑链接顺序表求q_{ij}的思路：

（1）非零元以行序为主序存放，即行号是递增存放的，且同行非零元按列号递增连续存放；

（2）存储结构带有各行非零元起始位置表rpos[]，可直接找任一行非零元；

（3）Q的元素q_{ij}是M的i行与N的j列的点积，需要一个累加器。

（4）M的第i行元素只参与求Q的第i行元素的点积，而且，M第j列的元素只与N的第j行的元素相乘。为Q设一行元素的累加器向量，整行元素的点积同时求。

例如，M的第1个非零元是1行1列的，
只能与N的第1行的非零元相乘，
并累加到Q的第1行元素累加器，
与N中与其相乘的非零元的列号决定累加器序号

算法5.5 稀疏矩阵相乘

$$M=\begin{bmatrix}3 & 0 & 0 & 5\\ 0 & -1 & 0 & 0\\ 2 & 0 & 0 & 0\end{bmatrix}\quad N=\begin{bmatrix}0 & 2\\ 1 & 0\\ -2 & 4\\ 0 & 0\end{bmatrix}\quad Q=\begin{bmatrix}0 & 6\\ -1 & 0\\ 0 & 4\end{bmatrix}$$

$$M \times N = Q$$

矩阵乘法规则：$q_{ij} = \sum_{k=1}^{n_1} m_{ik} \times nkj$

稀疏矩阵采用行逻辑链接顺序表存储结构，
需要相应的求q_{ij}的方法。

	i	j	e
1	1	1	3
2	1	4	5
3	2	2	-1
4	3	1	2

M->nzElem

	i	j	e
1	1	2	2
2	2	1	1
3	3	1	-2
4	3	2	4

N->nzElem

	i	j	e
1	1	2	6
2	2	1	-1
3	3	1	4

Q->nzElem

1	2	3	4
1	3	4	

M->rpos

1	2	3	4	5
1	2	3	5	

N->rpos

	1	2
ctemp	0	0

求Q的第1行

```
RLSMat MultSMatrix(RLSMat M, RLSMat N) {  // 算法5.5
  // 求稀疏矩阵乘积 Q=M×N，采用行逻辑链接存储表示。
  if(!M || !N ||M->n!=N->m) return NULL; // 矩阵相乘的合规检查
  RLSMat Q = allocRLSMat(M->m, N->n, 2*M->t); // 乘积矩阵Q初始化
  Q->t = 0;   int mi,ni,p,q,t,k,qj,tp,qtemp[30];
  if(M->t*N->t != 0) {            // Q是非零矩阵
    for(mi=1; mi<=M->m; ++mi) { // 处理M的每一行，当前是mi行
      for(k=1; k<=Q->n; ++i) qtemp[i] = 0;  // Q的mi行累加器清零
      Q->rpos[mi] = Q->t+1;     // 置Q的mi行的起始位置
      tp = M->rpos[mi+1];       // tp指向下一行的起始位置
      for(p=M->rpos[mi]; p<tp; ++p) { // 对当前行中每一个非零元
        ni=M->nzElem[p].j; // 取p元素对应元在N中的行号
        t = N->rpos[ni+1]; // t指向N的ni+1行的起始位置(判断ni行结束)
        for(q=N->rpos[ni]; q<t; ++q) { //N第ni行非零元都是p元对应元
          qj = N->nzElem[q].j;          // 乘积元素在Q中列号
          qtemp[qj] += M->nzElem[p].e*N->nzElem[q].e; //乘积累加
        }
      } // 循环结束时，求得Q中第mi行的非零元
      for(qj=1; qj<=Q->n; ++qj) // 压缩存储该行非零元
        if(qtemp[qj]) { // 非零元 q_ij 存入乘积矩阵Q的存储结构
          if(++Q->t > Q->size) reallocRLSMat(Q);  // 扩容
          Q->nzElem[Q->t].i = mi;
          Q->nzElem[Q->t].j = qj;
          Q->nzElem[Q->t].e = qtemp[qj];
        }
    } // for mi
    Q->rpos[mi] = Q->t+1;  // 置Q的末尾三元组的下一位置
  } // if
  return Q;
}
```

按上述分析形成的思路，设计了算法5.5

算法5.6 构建稀疏矩阵的十字链表

前面讨论了稀疏矩阵的顺序存储结构：

三元组顺序表和行逻辑链接的顺序表

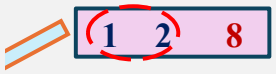
算法5.4和5.5实现了稀疏矩阵快速转置和相乘。

对于其他操作和应用，是否也能高效实现呢？

例如，矩阵M的三元组顺序表，

观察M上的非零元分布改变的情形。

增加非零元（1, 2, 8）



非零元（2, 2, -1）的值
被改变为0，需删除

非零元的新增和删除平均需要移动三元组表的一半非零元。
非零元频繁移动会导致相关应用的效率低下。

顺序存储结构不便于进行插入和删除的操作，

当稀疏矩阵中非零元分布经常变化时，采用链式存储结构。

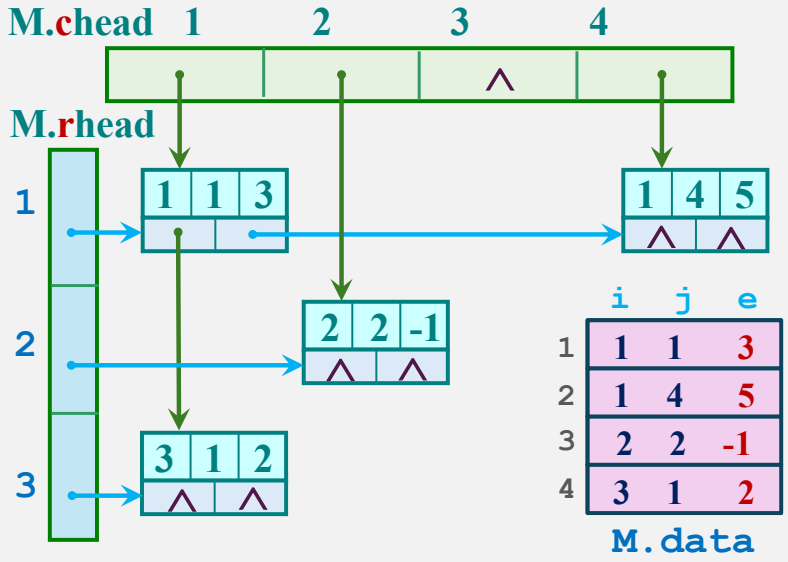
为了维护矩阵元素之间的同行、同列关系，

在表示非零元的结点中设立两个指针：

一个指向同一行的下一个非零元，

一个指向同一列的下一个非零元。

通常称这样的存储结构为十字链表。



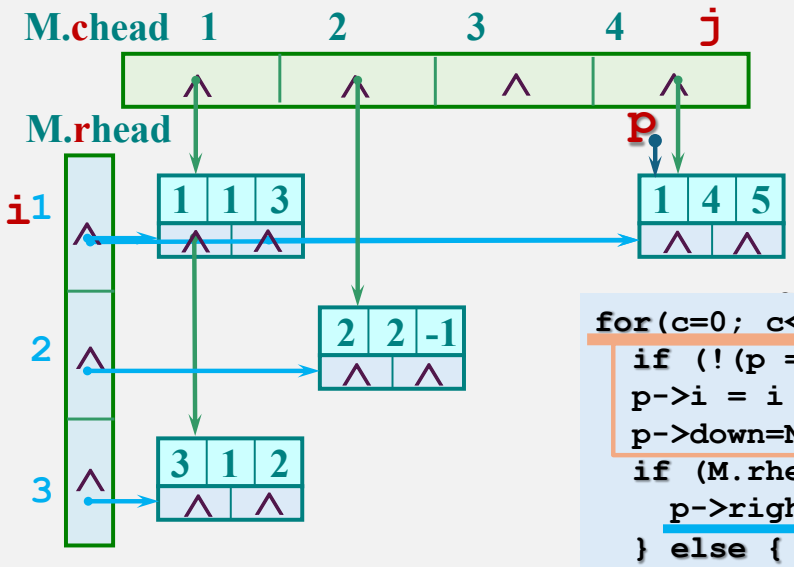


图5.6的稀疏矩阵3行4列，4个非零元的行、列号和非零值可由以下3个数组传递给算法。

```
int is[]={1, 2, 1, 3};
int js[]={4, 2, 1, 1};
int es[]={5, -1, 3, 2};
```

算法调用：

```
CreateSMatrix_OL(M, 3, 4,
                 4, is, js, es);
```

算法5.6 构建稀疏矩阵的十字链表

算法的核心部分是由参数提供的数据，构建 t 个非零元结点，并插入十字链表。实际上就是插入到无头结点的行表和列表。
先在行表中找到插入位置，并插入。
再在列表中找到插入位置，并插入。

```
for(c=0; c<t; c++) { // 构建 t 个非零元结点，并插入十字链表
  if (!(p = (OLNode *)calloc(1, sizeof(OLNode)))) return ERROR;
  p->i = i = is[c];  p->j = j = js[c];  p->e = e = es[c];
  p->down=NULL;  p->right=NULL; // 生成结点
  if (M.rhead[i] == NULL || M.rhead[i]->j > j) {
    p->right = M.rhead[i];   M.rhead[i]= p;    // 在行表头插入
  } else {  // 寻查在行表中的插入位置
    for(q=M.rhead[i]; (q->right) && q->right->j < j;  q=q->right);
    p->right = q->right;  q->right = p;
  }  // 完成行插入
  if (M.chead[j] == NULL || M.chead[j]->i > i) {
    p->down = M.chead[j];   M.chead[j]= p;     // 在列表头插入
  } else {  // 寻查在列表中的插入位置
    for (q=M.chead[j]; (q->down) && q->down->i < i;  q=q->down);
    p->down = q->down;  q->down = p;
  }  // 完成列插入
} // for
return OK;
} // CreateSMatrix_OL
```

注意：(1) 十字链表的行、列链表都是无头结点单链表
(2) 构建十字链表与非零元的插入次序无关

算法5.8 求广义表深度

广义表 *LS* 深度 Depth(*LS*) 的递归定义

基本项：Depth(*LS*) = 1　　当*LS*为空表时

Depth(*LS*) = 0　　当*LS*为原子时

归纳项：Depth(*LS*) = 1+Max{Depth(α_i)}

$1 \le i \le n$　$n \ge 1$

递归的算法5.7就是这个递归定义的自然表达。

以图5.13的广义表和存储结构作为演示例子

D = ((),(e),(a,(b,c,d)))

从用嵌套括号表达的逻辑结构看，

广义表的深度的深度就是它的括号重数。

从存储结构看，深度就是表结点的层数。

3

与定义对应，算法遇空表返回1，遇原子返回0；

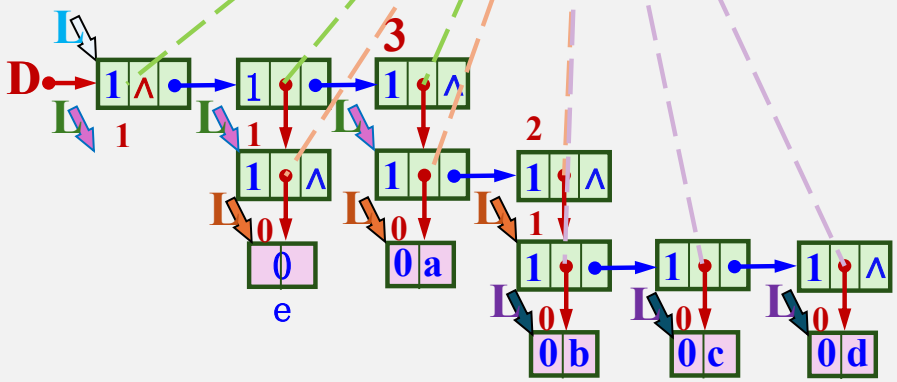
非空表则递归求每个子表深度，返回最大值加1。

下面观察调用算法求广义表D的深度d的过程：

d = GListDepth(D);

```
int GListDepth(Glist L) { // 求广义表L的深度
  int max, dep;
  if(!L) return 1;  // 空表返回深度 1
  if(L->tag==ATOM)return 0;  // 原子返回 0
  for(max=0; L; L=L->ptr.tp){ // 对子表递归
    dep = GListDepth(L->ptr.hp);
    if(dep>max)max = dep; //求子表最大深度
  }
  return max+1; // 返回深度
}
```

值参 L 可作为工作指针，可对L赋值，但不能对L指向的结点各域赋值！



算法5.9　复制广义表

对广义表的递归结构的分析有两种典型视角。

1. 非空表中的元素可以是表。
2. 非空表可划分为表头和表尾，均可以是表。

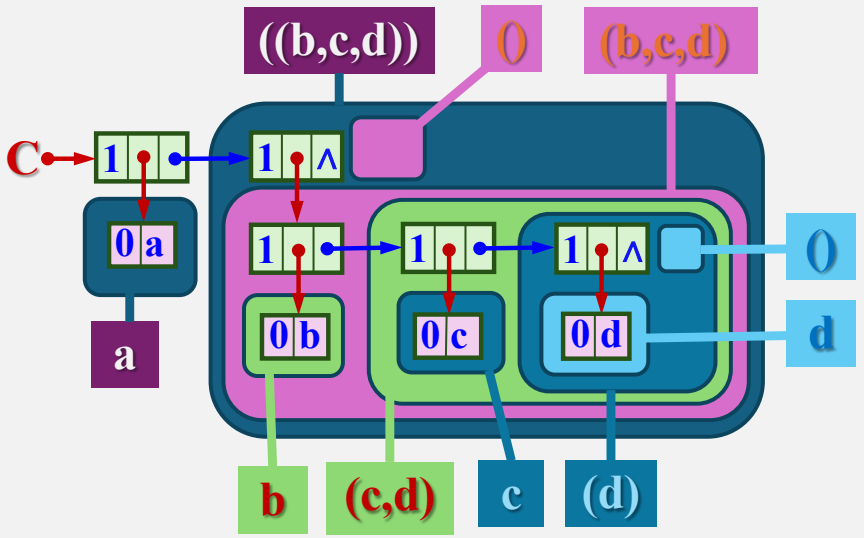
算法5.8采用第1种视角，依次对元素递归求深度，然后综合得到表的深度。

算法5.9则采用第2种视角，对表头和表尾分别递归复制给定的广义表，完成表的复制。

以图5.9的广义表C为例，其逻辑序列和存储结构分别为：

C = “(a,(b,c,d))”

我们看其表头和表尾的递归划分的演示。



算法5.9　复制广义表

假设算法外部定义了两个广义表变量：

GList C, S;

且C已构建。现要由C复制一个与C相同的广义表S。

要区分复制与赋值的不同：

赋值：　S = C;　这令S也指向C的头结点，共享C的存储结构。

复制：　CopyGList(C);

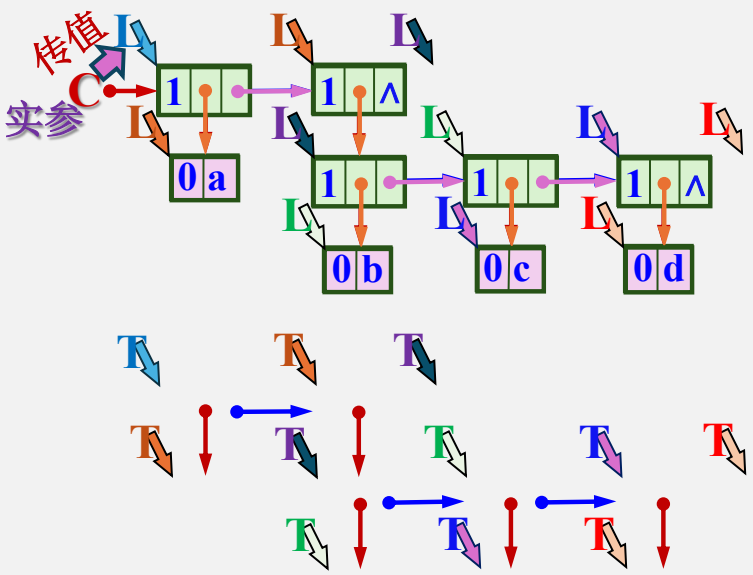
则是构建并返回与C相同的存储结构T，T是独立的广义表。

传值

```
GList CopyGList(GList L) {  // 算法5.9 复制广义表
  Glist T;
  if (!L) T = NULL;    // 复制空表（空指针）
  else {
    if (!(T=(GList)malloc(sizeof(GLNode))))
      exit(OVERFLOW);
    T->tag = L->tag;
    if (L->tag==ATOM)T->atom = L->atom; //复制原子
    else {
      T->ptr.hp = CopyGList(L->ptr.hp); //复制表头
      T->ptr.tp = CopyGList(L->ptr.tp); //复制表尾
    }
  }
  reruen T;
}
```

算法5.9采用表头表尾分别递归复制的方式，复制广义表的存储结构。

下面用不同的颜色的算法框背景和指针演示算法对广义表的复制过程。



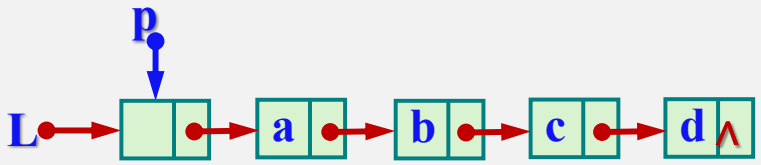
首先，复习构建单链表的算法2.11。
假定元素序列是：s="a, b, c, d"，看算法过程的演示。
结论：类似链队列的尾指针，只要指针 p 总是指着表尾结点，只需这个指针，即可实现在表尾插入结点。
其次，算法5.8是按表头表尾分别递归复制给定的广义表。而现在讨论的算法5.9则是按广义表的逻辑序列，构造广义表。这至少有两点学习意义：
1. 可作为比较，采用与算法5.8的不同视角，以逐个对元素分层递归的方式构建广义表。
2. 从广义表的逻辑序列提取广义表的结构和元素信息，是进一步学习和提高对字符串操作和处理能力。
以图5.9的广义表C为例，其逻辑序列为：

C = "(a,(b,c,d))"

为了简明，约定广义表的原子为单个字母，且逻辑序列是正确的，忽略对错误的判断。
每一对括号表示广义表结构的一层，两重括号表示该表是两层结构，深度为2。逗号分隔了元素。因此，在提取广义表信息时，要能够做到：
(1) 识别左右括号，去除一重括号；
(2) 识别逗号，将当前串分割为头尾子串；
(3) 判别单个字母，即识别原子。

算法5.10和5.11　构建广义表的存储结构

```
char *s = "abcd";
```



初始建头结点

```
p->next = (LNode*)calloc(sizeof(LNode));
p = p->next;
```

依次在表尾插入各元素结点

"(a,(b,c,d))" ⇒ "a,(b,c,d)"
"a,(b,c,d)" ⇒ "a" 和 "(b,c,d)"
"a"

对算法5.11作一个简化，只保留从S串提取广义表的结构和原子信息的相关操作。如以下算法GetGListInfo(S)所示。

图5.10的广义表C的逻辑序列为：

S = “(a,(b,c,d))”

以下调用GetGListInfo(S)， 演示从串S提取广义表C的结构和原子信息的递归过程。

算法框底色的变化对应算法递归调用和返回的当前层次。

算法5.10和5.11如何提取广义表的结构信息？

算法GetGListInf(S)实际上就是算法5.10主框架。能提取表结构信息，构建广义表就水到渠成了。

S hsub sub

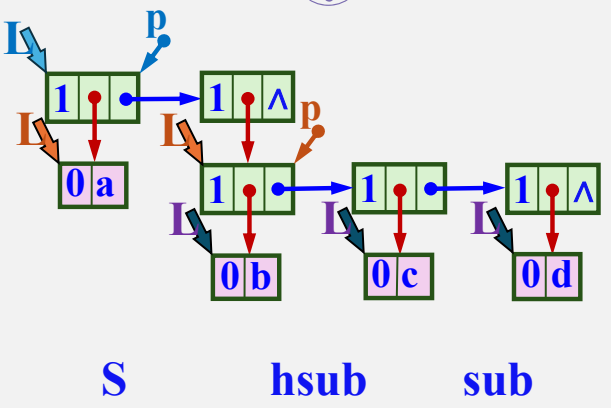
```
GetGListInfo(SStr S) {
  if (SStrCmp(S, emp)) L = NULL;   // S为"()"，是空表，emp="()"
  else {
    if (SStrLen(S)==1) 是原子;   // 当前串S只有1个字符
    else {
      sub=SStrSub( S, 2, SStrLen(S)-2);   // 得到脱外层括号的子串sub
      do {   // 依次分离子表
        hsub=Sever(sub);   // 从sub中分离出表头串 hsub，sub保留表尾
        GetGListInfo(hsub);        // 对表头串递归分解
      } while (!SStrEmpty(sub));   // 表尾串sub不空，继续
    } // else
  } // else
}
```

算法结束

算法5.10 和 5.11　构建广义表的存储结构

```
GList NewGList(SString S) {  // 算法5.10，构建广义表
  SStr sub, hsub, emp = SStrNew("()");
  GList L, p, q;
  if (SStrCmp(S, emp)) return NULL;  // S为"()"，是空表，emp="()"
  if(!(L=(GList)malloc(sizeof(GLNode)))) exit(OVERFLOW);
  if (SStrLen(S)==1){ L->tag = ATOM;  L->atom = S[1]; } //置为原子
  else {  L->tag = LIST;   p = L;      // 非空表的表结点
    sub=SStrSub(S, 2, SStrLen(S)-2);  // 得到脱外层括号子串sub
    do {  // 依次分离子表
      hsub = Sever(sub);  // 从sub中分离出表头串 hsub，sub保留表尾
      p->ptr.hp = NewGList(hsub);   q = p;   // 递归建子表
      if (!SStrEmpty(sub)) {
        if (!(p=(GList)malloc(sizeof(GLNode)))) exit(OVERFLOW);
         p->tag = LIST;   q->ptr.tp = p;
      }
    } while (!SStrEmpty(sub));  // 表尾串sub不空，则循环继续
    q->ptr.tp = NULL;
  } // else
}
```

算法里棕红色语句构建广义表的存储结构。
下面快速浏览算法5.7的运行演示。



算法结束